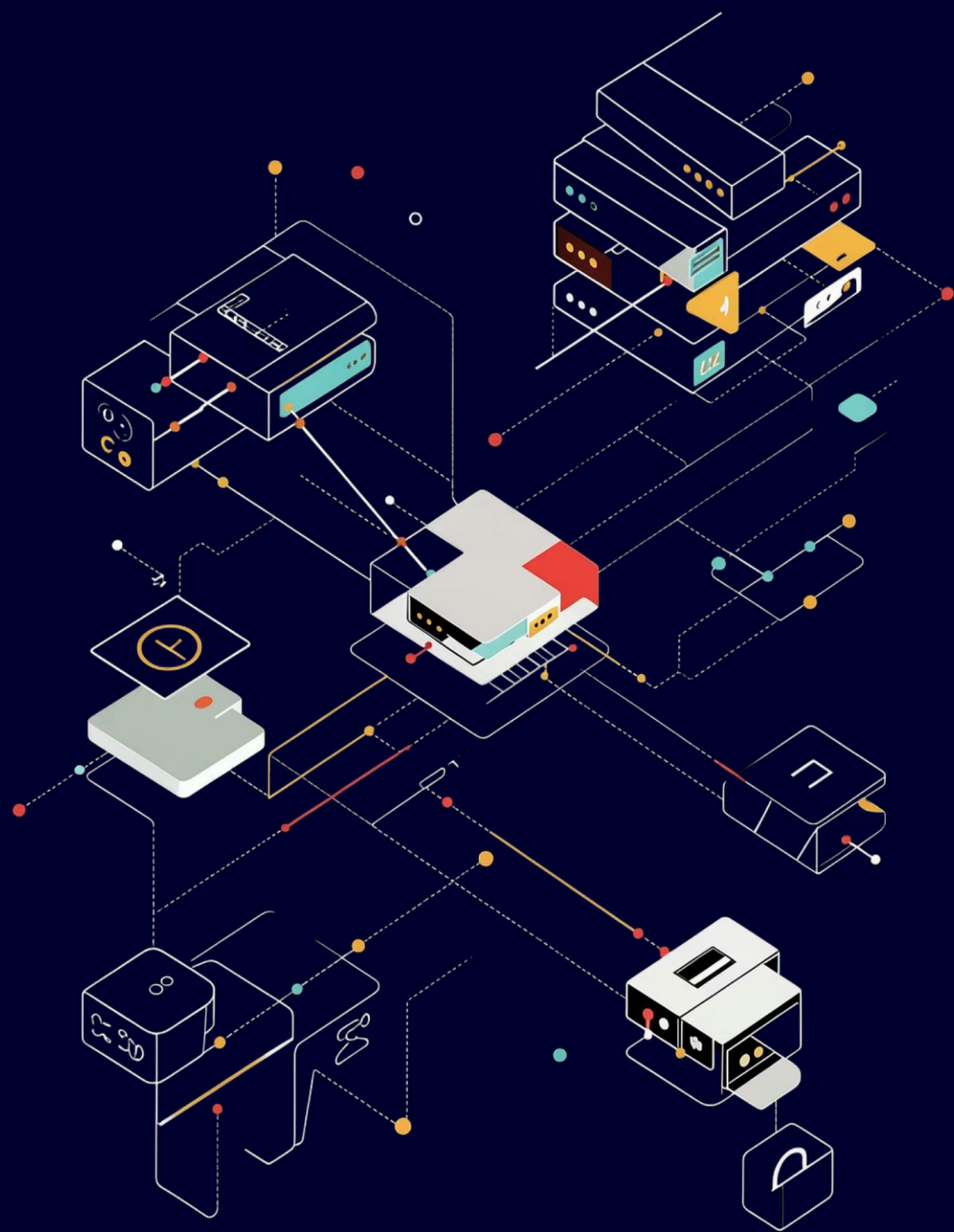
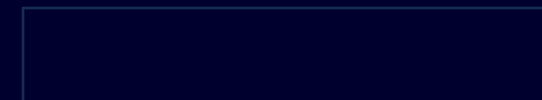




Глубокое сравнение и клонирование объектов

От equals до собственной сериализации



Проблема стандартного equals

Реализация по умолчанию

Класс Object использует сравнение по адресам в памяти:

```
public boolean equals(Object obj) {  
    return (this == obj);  
}
```

Проблема: Сравниваются только адреса в памяти. Объекты с одинаковыми полями **≠ равны**

Пример

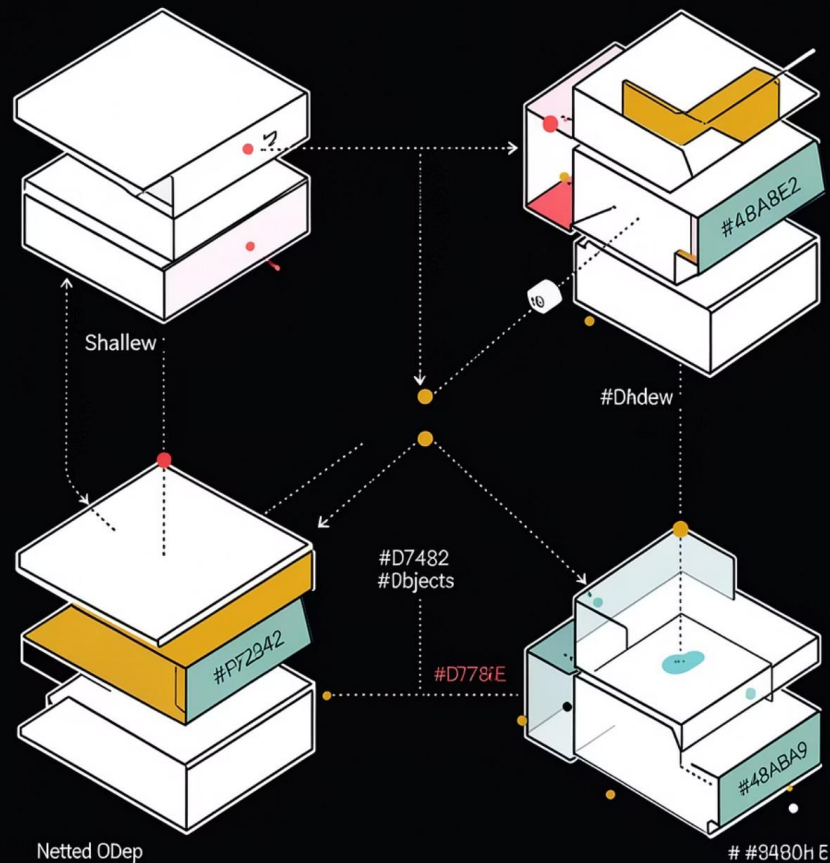
```
User user1 = new User("Алексей", 25);  
User user2 = new User("Алексей", 25);  
  
user1.equals(user2); // false !!!
```

Последствия:

- HashSet хранит дубликаты
- contains() не находит объект
- indexOf() возвращает -1

 Вывод: equals() нужно ПЕРЕОПРЕДЕЛЯТЬ для сравнения по содержимому

Глубокое сравнение (Deep equals)



Поверхностное (shallow)

1. Сравниваются только прямые поля
2. Для объектов — сравниваются ссылки
3. Вложенные объекты не обходятся

Глубокое (deep)

1. Рекурсивный обход всех полей
2. Для каждого вложенного объекта вызывается его equals()
3. Сравнивается полное содержимое

i Глубокое сравнение – рекурсивное сравнение всех полей, включая вложенные объекты

Проблема циклов при глубоком сравнении

Схема работы с отслеживанием посещённых объектов

01

Создать Set visited

02

Перед рекурсией проверить: объект уже в visited?

03

Если да → вернуть true (остановить обход)

04

Если нет → добавить в visited и продолжить

Проблема без visited:

$A \rightarrow B \rightarrow C \rightarrow A$ (цикл)

При рекурсивном обходе → бесконечная рекурсия →
StackOverflowError

Решение с visited:

Отслеживание посещённых объектов

Без visited – StackOverflowError. С visited – безопасное
сравнение графов с циклами

Сравнение коллекций и массивов

Тип массива	Способ	Что происходит
Одномерный	<code>arr1.equals(arr2)</code>	Сравнение ссылок
Одномерный	<code>Arrays.equals(arr1,arr2)</code>	Поэлементное сравнение
Многомерный	<code>Arrays.equals(deep1,deep2)</code>	Только первый уровень
Многомерный	<code>Arrays.deepEquals(deep1,deep2)</code>	Рекурсивное сравнение

Коллекции (List, Set, Map)

List

С учётом порядка элементов

Set

Без учёта порядка

Map

Сравнивает ключи и значения



Важно: Элементы коллекций должны иметь корректно переопределённый `equals()`!

Сериализация как способ сравнения

Идея

Если байтовые массивы двух объектов совпадают → объекты равны

Почему это работает?

Сериализация фиксирует полное состояние объекта:

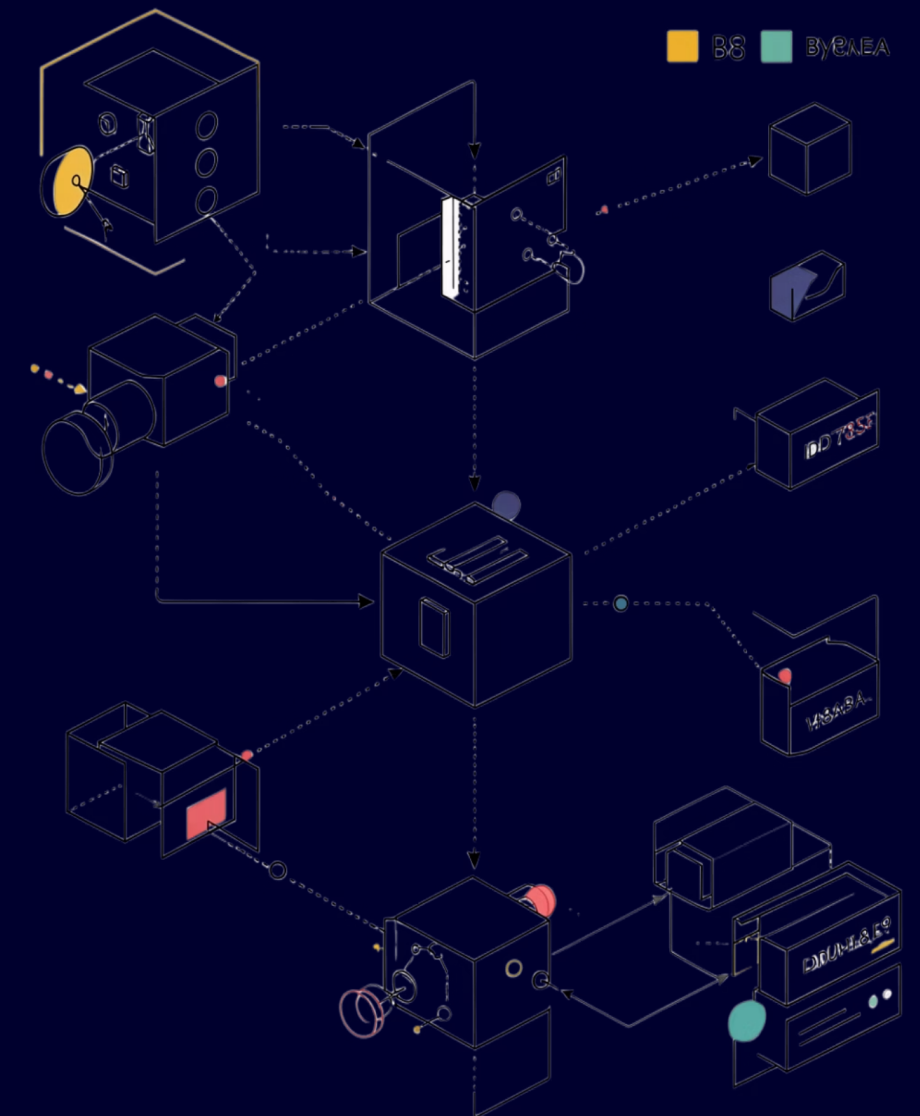
- Значения всех примитивных полей
- Содержимое всех вложенных объектов (рекурсивно)
- Элементы массивов и коллекций
- Структуру связей между объектами

Минусы

- Требуется implements Serializable
- Медленнее рекурсии

Плюсы

- Не нужно писать рекурсивный обход
- Автоматическая работа с циклическими ссылками
- Не нужно думать о типах полей



 Сериализация "раскладывает" объект на плоский поток байт

Реализация deepEquals через сериализацию



serialize()

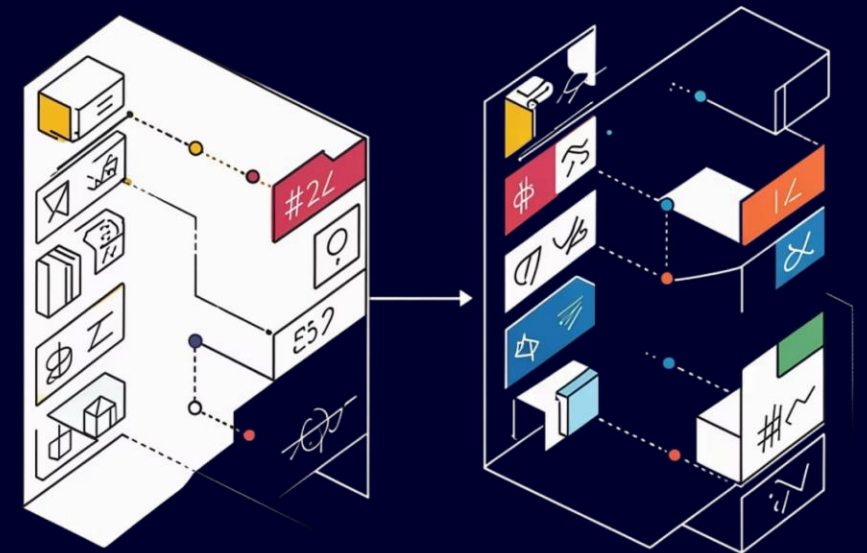
Записывает объекты в ByteArrayOutputStream через ObjectOutputStream

Arrays.equals()

Поэлементно сравнивается два байтовых массива

```
static boolean deepEqualsViaSerialization(Object o1, Object o2) {  
    byte[] b1 = serialize(o1);  
    byte[] b2 = serialize(o2);  
    return Arrays.equals(b1, b2);  
}
```

⚠ Важно: Объекты должны реализовывать интерфейс Serializable



Сравнение производительности

Критерий	Рекурсия	Сериализация
Скорость	Высокая	Медленнее в 10-15 раз
Память	Мало	Много
Циклические ссылки	Нужно обрабатывать	Автоматически
transient поля	Сравниваются	Игнорируются
Требования	Нет	Serializable
Код короче и проще	—	✓
Меньше шансов ошибиться	—	✓
Не требует ручной обработки циклов и null	—	✓



Проблема глубокого копирования (clone)

Подход	Глубокое копирование	Работа с циклами
default clone()	✗	✗
Копирующий конструктор	✓	✗
Ручной deepCopy()	✓	✗
Сериализация	✓	✓

Проблема стандартного clone()

Делает ПОВЕРХНОСТНУЮ копию (shallow copy)

Оригинал и копия разделяют ссылки на объекты

Изменение копии → изменение оригинала

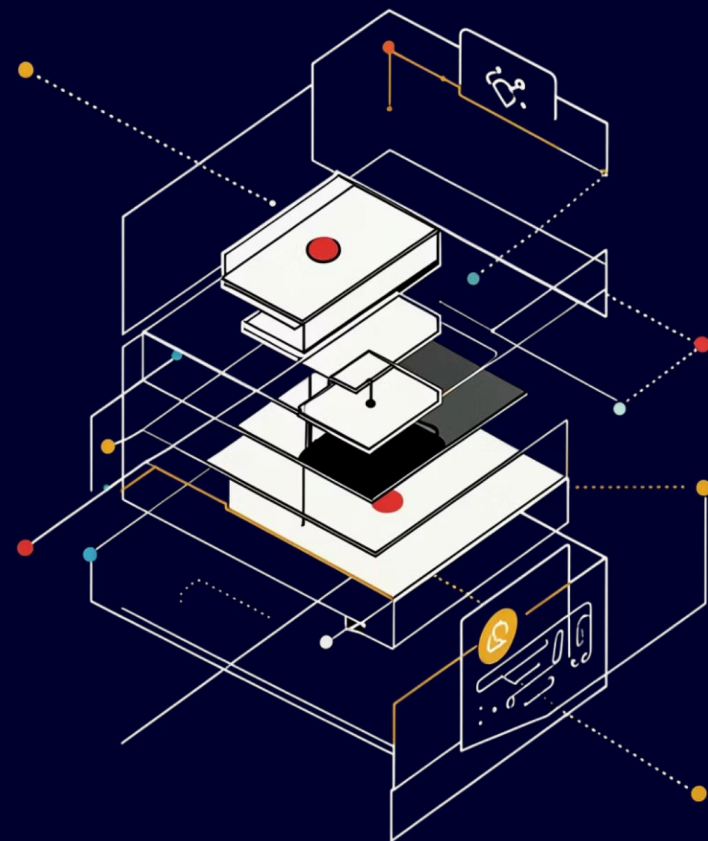
Почему Cloneable — «сломанный» интерфейс:

- Маркерный интерфейс (без метода clone())
- Метод clone() в Object — protected
- Бросает CloneNotSupportedException
- Не вызывает конструктор
- Проблемы с final полями

✓ Рекомендация: Используйте копирующий конструктор — это лучшая практика

Глубокое копирование через сериализацию

Метод клонирования



```
static <T> T deepCopyViaSerialization(T obj) {  
    byte[] bytes = serialize(obj);  
    return deserialize(bytes);  
}
```

Плюсы:

- Одна строка кода
- Копируется весь граф объектов
- Не нужно писать рекурсию

Минусы:

- Медленно
- Требуется Serializable
- Нужен конструктор без параметров